

NETRIK HACKATHON 2026

Loan Risk Prediction System – Documentation

1.Introduction

The loan prediction System is a web-based application developed to predict whether a loan application will be approved or not approved based on applicant details.

This system integrates:

- Frontend(HTML, CSS, JavaScript)
- Backend API(FastAPI)
- Machine learning model(Random Forest)
- Data Processing pipeline(Python)

Additionally, the system provides a reason-based response, explaining why a loan was approved or rejected.

2. Objective :

The main objectives of this project are :

- To automate the loan approved prediction process
- To build a user-friendly web interface
- To preprocess and clean real-world loan datasets
- To train a machine learning classification model
- To integrate frontend and backend using REST API

- To provide both prediction and reason for decision

3. System Architecture :

The system consists of two major parts:

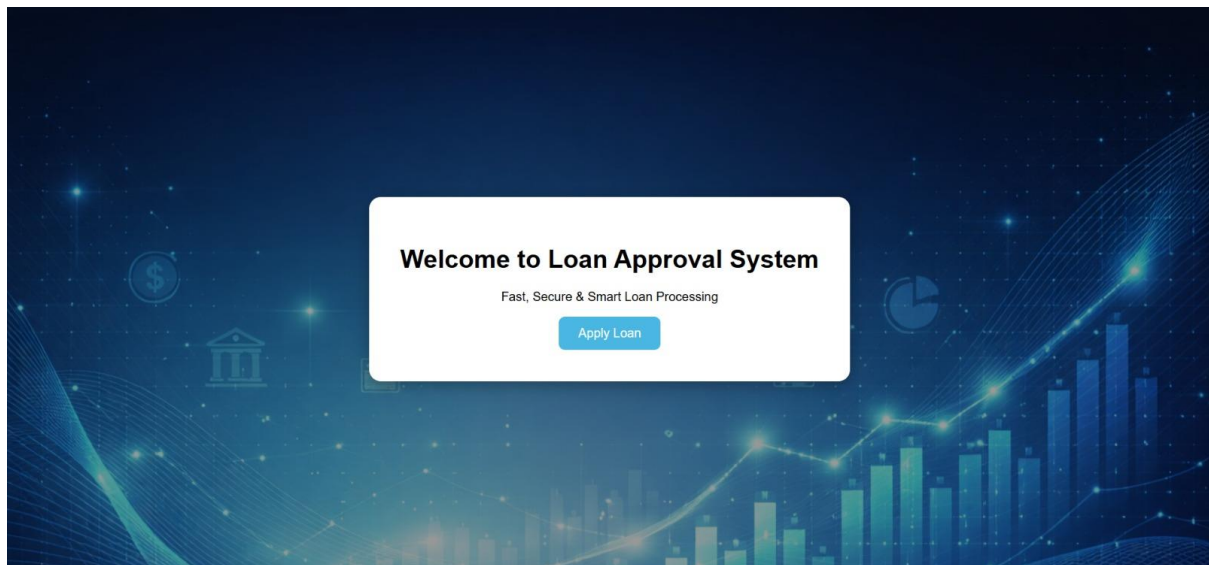
Frontend(User Interface)

Developed using :

- HTML
- CSS
- JavaScript

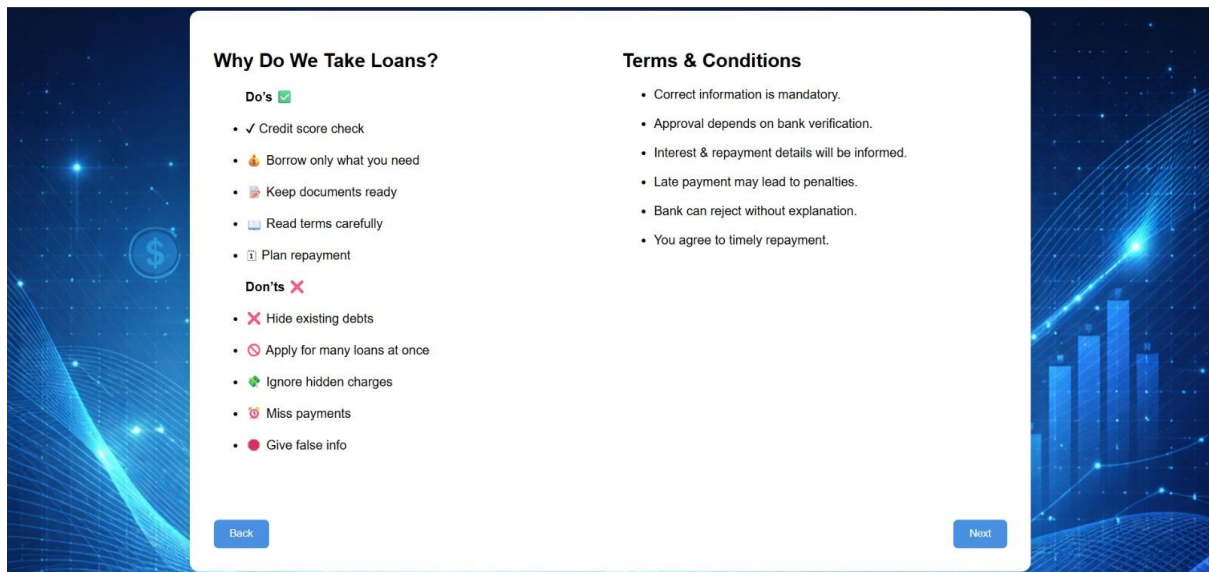
The frontend contains four connected pages:

1 1.home.html



- Displays welcome message
- Background image added
- “Apply Loan” button redirects to terms page

2 2.terms.html



- Displays Do's and Don'ts of Loan

- Displays Terms & conditions

- Navigation buttons:

->Back -> home.html

->Next -> cal.html

3.cal.html

The screenshot shows a 'Loan Calculator' web interface. It features a central white panel with a dark blue background. The panel has a title 'Loan Calculator' with a house icon. Below the title are three input fields: 'Loan Amount (₹):' with the value '500000', 'Months:' with the value '6', and 'Interest Rate (% per year):' with the value '0.9'. Below these fields are three buttons: 'Calculate', 'Next', and 'Skip'. At the bottom of the panel, under the heading 'Loan Details:', there is a grey box containing the following text: 'Monthly EMI: ₹83552.22', 'Total Payment: ₹501313.32', and 'Total Interest: ₹1313.32'. The background of the entire page is a dark blue gradient with a glowing bar chart and a line graph.

- Calculators EMI

- Inputs :
 - >Loan Amount
 - >Number of months
 - >Interest Rate
- Displays:
 - >Monthly EMI
 - >Total payment
 - >Total Interest
- User must calculate before proceeding
- Redirects to man.html

4.main.html(Loan Eligibility Form)

The image shows a 'Loan Eligibility Form' centered on a blue background with financial icons like a dollar sign, a bank building, and a bar chart. The form has the following fields:

- Gender: Male (dropdown)
- Married: Yes (dropdown)
- Dependents: 1 (text input)
- Education: Graduate (dropdown)
- Self Employed: Yes (dropdown)
- Applicant Income: 100000 (text input)
- Co-applicant Income: 50000 (text input)
- Loan Amount: 75000 (text input)
- Loan Term (months): 9 (text input)
- Credit History: 1 (text input)
- Property Area (1 Rural, 2 Urban, 3 Semiurban): 2 (dropdown)

A blue button labeled 'Check Eligibility' is at the bottom of the form.

Collects applicant information:

- Gender
- Married
- Dependents

- Education
- Self Employed
- Applicant Income
- Coapplicant Income
- Loan Amount
- Loan Term
- Credit History
- Properly Area

After submission:

- Data is sent to backend using:

POST :- `http://127.0.0.1:8000/predict`

Result is shown as alert message.

4.Data Processing Pipeline(python)

Data Export (file.csv)

Dataset saved as CSV file using:
`df.to_csv(...)`

Data Loading ([data.py](#))

- Reads dataset using Pandas
- Displays first 5 rows

Data Merging ([Merge.py](#))

Three datasets are merged using Loan_ID :

- Dataset 1 -> Gender, ApplicationIncome
- Dataset 2 -> Credit_History, LoanAmount
- Dataset 3 -> Property_Area

Merged dataset saved as :
Merged.csv

Data Cleaning([clean.py](#))

Cleaning steps performed:

- Missing LoanAmount filled using median
- Missing credit_History filled using mode
- Duplicate records removed
- Cleaned file saved as:

Cleaned.csv

5. Model Training([Train.py](#))

Algorithm used :
Random Forest Classifier

Training Steps:

- Load dataset
- Remove Loan_ID column
- Fill missing numeric values using median
- Fill missing categorial values using mode

- Convert Loan_Status:
Y -> 1
N -> 0
- Apply one-Hot Encoding using get_dummies()
- Split dataset:
80% Training
20% Testing
- Train RandomForestClassifier
- Evaluate using:
Accuracy
Confusion Matrix
Classification Report
- Save:
Loan_model.pkl
Loan_column.pkl

6.Backend API([Loan.py](#))

Developed using:

- FastAPI
- Pydantic
- Pandas

- Joblib
- Asyncio
- ThreadPoolExecutor

CORS enabled for frontend-backend communication.

API is running

GET “/”

Response:

Loan Prediction API Running

POST “/predict”

Accepts applicant data in JSON format

Exampe Input:

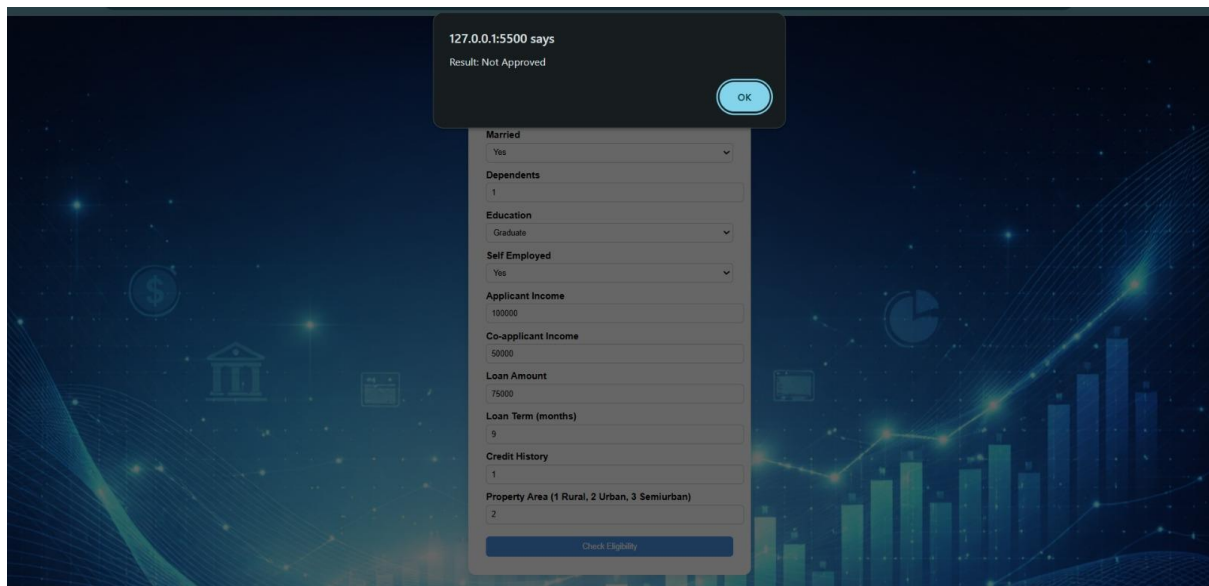
```
{
  "Gender": 1,
  "Married": 1,
  ..
}
```

Prediction Logic

Steps:

- Convert input dictionary into DataFrame
- Apply one-hot encoding
- Reindex columns using saved column list
- Predict using trained model
- Generate result

Reason-Based Output(Important Update)



The system not only predicts approval status but also gives a reason

If Approved:

- Returns:

Status: Approved

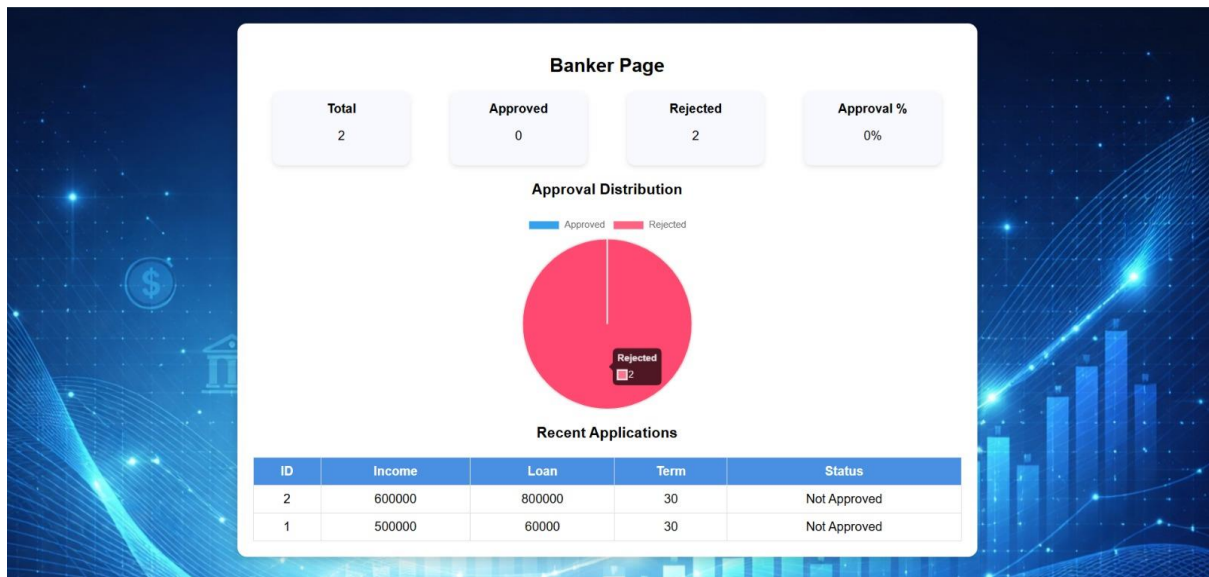
Reason: Strong financial profile and good credit history

If Not Approved:

- System checks conditions like:
 - > Poor credit history
 - > Low applicant income (< 2500)
 - > High loan amount (>250)
 - > Very long loan term (> 360 months)
- Returns:
 - > Status: Not Approved
 - > Reason: List of detected risk factors

This makes the system more transparent and user-friendly.

7.Working Flow



This page is used by banker/admin to verify and monitor loan applications.

User ->
 Home Page ->
 Terms Page ->
 Loan Calculator ->
 Backend API ->
 Machine Learning Model ->
 Prediction + Reason ->
 Result Displayed

8. Technologies Used

Component

Frontend
 Backend
 ML Model
 Data Processing
 Model Storage
 Async Handling

Technology

HTML, CSS, JavaScript
 FastAPI
 Random Forest
 Pandas
 Joblib
 Asyncio + ThreadPoolExecutor

9. Key Features

- Multi-page interactive frontend
- EMI calculator before eligibility check
- Data merging and cleaning pipeline

- Random Forest Classification model
- REST API integration
- Async prediction handling
- Reason-based loan display

10.Conclusion

The loan prediction system successfully integrates frontend development, backend API, and machine learning into a complete working application.

The system automates loan eligibility checking and provides both prediction and reason, making the decision process transparent and efficient.

This project demonstrates practical implementation of:

- Data preprocessing
- Machine learning model training
- REST API development
- Frontend-backend integration